Lecture 7 (Part 1): Model Selection & Regularization (ISLR Ch. 6)

Selecting predictors and tuning regularization with cross-validation

FNCE 5352 — Financial Programming & Modeling (R Weeks 1–7)

2026-01-01

Tonight’s plan (Lecture 7)

- Part 1: ISLR Ch. 6 — selecting predictors + regularization
- Part 2: ISLR Ch. 7 — moving beyond linearity
- Finish: ~30 min credit scoring project walkthrough

Why we care about model selection

- In finance, we *always* have more potential predictors than we should trust
- More predictors ⇒ more flexibility ⇒ easier to overfit
- Also: interpretability matters (governance, auditability, model risk)

Two different goals

- **Prediction:** lower out-of-sample error
- **Interpretation:** understand which predictors matter (direction + magnitude)
- Often a trade-off: the best predictive model may be hard to explain

The core problem: p feels large

- Suppose we have p candidate predictors
- If we try *all* subsets: 2^p models (explodes fast)
- Even if you can compute them, choosing among them is the real challenge

Three families of solutions (Ch. 6)

1. **Subset selection** (choose predictors, fit least squares / GLM)
2. **Shrinkage / regularization** (keep predictors, shrink coefficients)
3. **Dimension reduction** (not our focus tonight)

Train/test + CV reminder (from Lecture 6)

- One split is noisy
- CV estimates out-of-sample performance more reliably
- We'll reuse CV to pick: model size, λ, polynomial degree, spline df

Subset selection: the idea

- Goal: find a subset of predictors that “works well”
- Fit the model *only* on that subset
- Then evaluate / compare candidate subsets

Best subset selection (conceptual)

- For each model size k (0, 1, 2, ..., p):
  - search all subsets with k predictors
  - keep the best one (lowest RSS / highest likelihood)
- Then choose k using a selection rule (CV, AIC/BIC, etc.)

Stepwise selection (practical search)

- **Forward stepwise:** start empty, add the best next predictor each step
- **Backward stepwise:** start full, remove the worst predictor each step
- **Hybrid:** allow adds/removes

What stepwise is (and is not)

- It's a **greedy search** (locally optimal choices)
- It may miss the best subset
- But it's often "good enough" and very interpretable

How do we choose model size k?

- We need a rule for “best” complexity
- **Option 1:** CV estimate of test error
- **Option 2:** information criteria (AIC/BIC)
- **Option 3:** adjusted R^2 (regression only)

Adjusted R^2 (quick intuition)

- R^2 always increases when you add predictors
- Adjusted R^2 adds a penalty for complexity
- So it can go up or down when you add a variable

AIC / BIC (quick intuition)

- Start with goodness-of-fit (likelihood)
- Subtract a penalty for number of parameters
- BIC penalizes complexity more than AIC

CV for model size: the clean story

- For each model size k:
  - pick a specific subset (best/stepwise) *within each fold*
  - evaluate on the fold's assessment set
- Choose k with the lowest CV error

Warning: selection can leak

- If you pick predictors using the full dataset, then CV the chosen model:
- you’re leaking information across folds
- CV error will look better than it truly is

Subset selection in practice (what we'll do in demo)

- Use `leaps::regsubsets()` to run forward/backward stepwise
- Look at metrics across model sizes (RSS, adj R^2, BIC)
- Optionally: wrap in a simple CV loop for model size

Regularization: why shrink instead of drop?

- When predictors are correlated, least squares coefficients can be unstable
- Shrinkage trades a little bias for a lot less variance
- Often improves out-of-sample performance

Bias–variance framing (again)

- More flexibility ⇒ lower bias, higher variance
- Regularization intentionally increases bias
- ...to reduce variance and improve test performance

Ridge regression: the idea

Optimization problem:

$$\text{minimize} \quad \text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2$$

Intuition: the budget metaphor**

- Think of λ as a "budget" for coefficient spending
- Each unit of $|\beta_j|$ costs you: the squared amount goes against your budget
- Ridge penalizes *large* coefficients: $\beta_j = 0.1$ costs 0.01; $\beta_j = 1.0$ costs 1.00
- So ridge "spreads" the weight around: better to use many small coefficients than a few huge ones
- When $\lambda \uparrow$ (budget shrinks): all coefficients shrink toward 0 (usually not exactly 0)

Lasso: the idea

Optimization problem:

$$\text{minimize} \quad \text{RSS} + \lambda \sum_{j=1}^{p} |\beta_j|$$

Intuition: same budget, different penalty**

- Lasso also has a budget λ, but it penalizes *absolute values* (L1)
- Ridge penalizes squared: $\beta_j = 0.1$ costs 0.01; Lasso penalizes linear: $\beta_j = 0.1$ costs 0.1
- **Result:** Lasso prefers *sparse* solutions (exact zeros)
  - It's cheaper to drop a predictor entirely than shrink it a little bit
  - With high λ: many coefficients snap exactly to 0 (automatic feature selection)

Why Lasso picks sparsity:

- Ridge budget: spread weight thinly (many small coefficients)
- Lasso budget: spend heavily on a few, or go to zero (binary decision)

Ridge vs lasso with correlated predictors

- If predictors are strongly correlated:
  - **Ridge** tends to share weight across them
  - **Lasso** tends to choose one and zero out others
- In finance: correlated ratios/macros are common → ridge can be very stable

The tuning knob: λ

- $\lambda = 0 \Rightarrow$ ordinary least squares / MLE
- Large $\lambda \Rightarrow$ heavy shrinkage $\Rightarrow$ underfit
- So we need to *tune* λ using CV

Standardization matters

The budget principle: units shouldn't matter

- Remember: regularization penalizes coefficient magnitudes using your "λ budget"
- If we measure one predictor in centimeters and another in miles, the "cost" is unfair
- Example: $\beta_{\text{centimeters}} = 100$ (in budget) vs $\beta_{\text{miles}} = 0.001$ (tiny!)
- **Solution:** Standardize all predictors to mean 0, SD 1 before fitting
  - Now $|\beta| = 0.1$ means the same thing regardless of original units
  - Fair budget allocation

Why standardize *inside* folds?

- If you standardize on the full dataset, then fit/CV on folds: you leak information
- Each fold should “learn” its own mean/SD from training data only
- Otherwise: CV folds know the global reference point → overly optimistic error estimates

In practice:

- `glmnet` standardizes by default (good!)
- But you must understand the leakage principle if you write custom tuning code
- Same principle as scaling in KNN (Lecture 6)

Why we'll implement with `glmnet`

- It's the standard tool in R for ridge/lasso
- Efficient + stable implementations (coordinate descent)
- Built-in cross-validation via `cv.glmnet()`

Regularization for classification too

- Same idea works for logistic regression
- We penalize the log-likelihood instead of RSS
- In credit scoring: penalized logistic is a common baseline

Tuning λ with CV: the algorithm

Pseudocode: what `cv.glmnet()` does

```
For each lambda value (high to low):
    For each fold i = 1, 2, ..., 10:
        Fit ridge/lasso on train fold i (with this lambda)
        Score on assessment fold i
        Record error
    Average error across all folds
Plot CV error vs log(lambda)
Pick lambda with lowest average error (or 1-SE rule)
```

- **Concept:** "Which amount of shrinkage generalizes best?"
- **1-SE rule:** pick a slightly larger λ (more stable, simpler)

Choosing λ: min vs 1-SE

- λ_{min}: best mean CV performance
- λ_{1se} : more regularization, often simpler + more stable
- In governance-heavy settings, λ_{1se} is a defensible default

Interpretability vs prediction (practical guidance)

- If you need a simple, explainable model:
  - consider lasso (sparse) or stepwise (few variables)
- If you need robust prediction under correlation:
  - ridge is often strong
- Always validate with CV + a holdout test set

Common pitfalls checklist

- Leakage: feature selection/scaling done outside resampling
- Tuning too many knobs on the test set
- Treating selected-model p-values as “truth”
- Forgetting stability (different folds → different models)

Mini roadmap: what you should be able to do after Part 1

- Explain why more predictors can hurt out-of-sample performance
- Describe best subset vs stepwise selection
- Use CV to choose complexity (k or λ)
- Implement ridge/lasso with `glmnet` in R

Bridge to Part 2: same tuning philosophy everywhere

- Subset selection: CV to pick model size k
- Regularization: CV to pick λ (then use λ_{1se} for stability)
- **Next:** polynomials, splines, GAMs all have knobs tuned the same way
- The pattern is: **more flexibility + CV + test set** (no magic, just discipline)

PASTE SCREENSHOT HERE — ISLR Figure suggestions

- Ch. 6.6: illustration of ridge vs lasso shrinkage paths (compare L2 vs L1 geometry)
- Ch. 6.6.2: CV curve vs $\log(\lambda)$ for ridge/lasso (show min vs 1-SE)
- Ch. 6.2: stepwise selection algorithm and overview

(Add page numbers once you have your copy open.)